SQL TECHNICAL APPENDIX

NYC Ride Hailing Strategy Case Study Query Documentation

Sai Sourabh Akula

Database: nyc_rideshare | Engine: MySQL 8.0 | Tables: trips and taxi_zone_lookup

github.com/sourabhakula/nyc-rideshare-strategy

Section 1 Database Architecture

1.1 Tables and Relationships

The database nyc_rideshare contains two tables. All analysis is performed against the enriched view v_trips_enriched which joins these tables and precomputes all derived metrics.

Object	Description
trips	Primary fact table. Approximately 100K rows of December 2025 HVFHV trip records after data quality filtering. One row per completed trip.
taxi_zone_lookup	Dimension lookup table. Maps integer zone IDs to borough, neighborhood name, and service zone type. Joined twice once for pickup zone, once for dropoff zone.
v_trips_enriched	MySQL VIEW (not materialized table). Joins both tables, decodes platform identifiers, computes all derived metrics. All 11 analysis queries run against this view exclusively.

1.2 Primary Keys, Constraints, and Indexes

The raw TLC dataset does not include a native primary key. The TLC publishes trip records without a guaranteed unique row identifier. For this analysis we treat each row as a unique trip and do not enforce a primary key constraint on the trips table.

taxi_zone_lookup primary key: LocationID (integer). Every PULocationID and DOLocationID in trips is a foreign key reference to this table. We use LEFT JOIN (not INNER JOIN) to preserve trips where the zone ID may not have a matching lookup entry protecting against data loss from zone ID mismatches.

Recommended indexes for production use:

- INDEX ON trips(hvfhs_license_num) used in every WHERE clause for platform filtering
- INDEX ON trips(pickup_datetime) used in Query 08 for daily aggregation and Query 09 for date level analysis
- INDEX ON trips(PULocationID) used in all zone level queries (Q02, Q03, Q10)
- INDEX ON trips(base_passenger_fare) used in NTILE window function ordering (Q04)

For a 100K row dataset these indexes have modest performance impact. At the full 1M row dataset they become essential without them, view based queries on the unindexed table would take 30 to 60 seconds each.

1.3 View Architecture: Why a View, Not a Table

All analysis runs against v_trips_enriched, a MySQL view, rather than querying the raw trips table directly. This design decision was deliberate:

- Single source of truth: the TLC minimum pay formula ($0.82/mile + $0.57/minute) appears once in the view definition. If the rate changes, only one place updates.
- Clean separation: analytical queries focus on business questions; the view handles data engineering (type conversions, NULL handling, flag derivations).
- Self documenting: anyone reading a query that runs against v_trips_enriched can look at the view definition to understand exactly what every column means.
- Auditability: the data quality filters (zero miles, null datetimes, impossible wait times) are in the view, not scattered across individual queries. They apply consistently.

Performance Note on Views

MySQL does not materialize views by default. Every query reexecutes the view's SELECT against the underlying tables. For 100K rows this is acceptable (sub second queries). If scaling to 1M+ rows, consider: CREATE TABLE trips_enriched_mat AS SELECT * FROM v_trips_enriched; and adding indexes to that materialized table. Refresh it after each data load.

Section 2 The Enriched View

VIEW: v_trips_enriched

This is the foundation of the entire analysis. Every derived column is documented below with its formula, purpose, and any caveats. The view runs at query time against the raw trips and taxi_zone_lookup tables.

Platform Decoding

```
CASE
  WHEN t.hvfhs_license_num = 'HV0003' THEN 'Uber'
  WHEN t.hvfhs_license_num = 'HV0005' THEN 'Lyft'
  ELSE 'Other'
END AS company_name
```

Converts the raw TLC license number to consumer brand name. The WHERE clause at the bottom of the view restricts to HV0003 and HV0005 only, so 'Other' will never appear in practice. The ELSE branch is defensive coding that handles any unexpected license numbers without crashing.

Time of Day Classification

```
CASE
  WHEN HOUR(t.pickup_datetime) BETWEEN 6  AND 9  THEN 'AM Rush'
  WHEN HOUR(t.pickup_datetime) BETWEEN 10 AND 15 THEN 'Midday'
  WHEN HOUR(t.pickup_datetime) BETWEEN 16 AND 19 THEN 'PM Rush'
  WHEN HOUR(t.pickup_datetime) BETWEEN 20 AND 22 THEN 'Evening'
  ELSE                                                'Late Night'
END AS time_of_day
```

Late Night covers 23:00 to 05:59 via the ELSE clause any hour not caught by the four BETWEEN clauses. Classified by PICKUP hour, not request or dropoff. Five bands aligned with NYC MTA peak pricing windows.

Zone Lookup Double Join

```
FROM trips t
LEFT JOIN taxi_zone_lookup pu  ON t.PULocationID  = pu.LocationID
LEFT JOIN taxi_zone_lookup do_ ON t.DOLocationID  = do_.LocationID
```

The same lookup table joined twice under different aliases (pu for pickup, do_ for dropoff). LEFT JOIN ensures trips with unmapped zone IDs are retained with NULL zone columns. The double join enables simultaneous access to both origin and destination geography in a single row, which is required for corridor analysis in Query 03.

Wait Time

```
ROUND(
  TIMESTAMPDIFF(SECOND, t.request_datetime, t.pickup_datetime)
  / 60.0
, 2) AS request_to_pickup_min
```

TIMESTAMPDIFF returns an integer. Dividing by 60.0 (not 60) forces floating point division, preserving decimal minutes. The WHERE clause filters wait time to 0 to 7200 seconds before rows enter the view.

SLA Flag

```
CASE
  WHEN TIMESTAMPDIFF(SECOND, t.request_datetime, t.pickup_datetime)
       <= 300 THEN 1
  ELSE 0
END AS sla_5min_met
```

300 seconds equals 5 minutes. Binary 0 or 1 flag. AVG(sla_5min_met) multiplied by 100 equals SLA percentage in any GROUP BY segment.

Total Passenger Charge

```
ROUND(
  t.base_passenger_fare
  + COALESCE(t.tolls, 0)
  + COALESCE(t.bcf, 0)
  + COALESCE(t.sales_tax, 0)
  + COALESCE(t.congestion_surcharge, 0)
  + COALESCE(t.airport_fee, 0)
  + COALESCE(t.cbd_congestion_fee, 0)
  + COALESCE(t.tips, 0)
, 2) AS passenger_total_charge
```

COALESCE applied to all optional fee fields. base_passenger_fare has no COALESCE because it is required trips with NULL base fare are excluded by the WHERE clause before reaching this calculation.

TLC Minimum Pay Violation Flag

```
CASE
  WHEN t.trip_miles > 0 AND t.trip_time > 0 THEN
    CASE
      WHEN t.driver_pay <
           ROUND((t.trip_miles * 0.82) + (t.trip_time / 60.0 * 0.57), 2)
      THEN 1 ELSE 0
    END
  ELSE NULL
END AS below_tlc_minimum
```

Nested CASE for safety: outer CASE guards against zero values. Inner CASE implements the TLC formula: minimum = ($0.82 times miles) + ($0.57 times minutes). trip_time is in SECONDS so divide by 60.0 first. Result: 1 = violation, 0 = compliant, NULL = invalid input.

Data Quality WHERE Clause

```
WHERE
  t.pickup_datetime    IS NOT NULL
  AND t.dropoff_datetime   IS NOT NULL
  AND t.request_datetime   IS NOT NULL
  AND t.base_passenger_fare IS NOT NULL
  AND t.driver_pay         IS NOT NULL
  AND t.trip_miles         > 0
  AND t.trip_time          > 0
  AND TIMESTAMPDIFF(SECOND, t.request_datetime, t.pickup_datetime)
      BETWEEN 0 AND 7200
  AND t.hvfhs_license_num IN ('HV0003', 'HV0005')
```

Seven conditions applied. The BETWEEN 0 AND 7200 wait time filter excludes both negative waits (timestamp errors) and unrealistic waits above 2 hours. The platform filter restricts to Uber and Lyft only. All conditions must be true for a row to enter the view.

Section 3 Query Documentation

QUERY 01 Market Position Scorecard

What is the complete competitive position of Uber vs. Lyft across every KPI that matters for business strategy?

Stakeholders: ALL stakeholders this is the baseline query every deck references first

> **Why This Query Is First**
>
> You cannot evaluate borough or segment performance without knowing the platform baseline. Every finding in Q02 through Q11 is contextualized against these numbers. Q01 is slide 1 of every stakeholder deck.

Key Technique: Window Function for Market Share

```
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS market_share_pct
```

SUM(COUNT(*)) OVER () is a window function that computes the total count across ALL rows in the GROUP BY result set. This allows market share to be calculated in a single query without a subquery or self join. The result always sums to 100% regardless of platform count.

Key Technique: Aggregate Take Rate

```
ROUND(
  (SUM(base_passenger_fare) - SUM(driver_pay))
  / NULLIF(SUM(base_passenger_fare), 0) * 100
, 2) AS avg_take_rate_pct
```

NULLIF(SUM(base_passenger_fare), 0) prevents division by zero. The SUM based method is mandatory see Assumption A6 in the main document for why AVG of per trip take rates is mathematically wrong.

Expected Output Key Verified Numbers

Column	Expected Value (Dec 2025)
total_trips (Uber)	71,518
market_share_pct (Uber)	72.49%
avg_take_rate_pct (Uber)	17.95%
avg_take_rate_pct (Lyft)	19.75%
sla_5min_pct (Uber)	59.80%
below_tlc_min_pct (Uber)	1.05%
below_tlc_min_pct (Lyft)	0.02%

QUERY 02 Demand Heatmap

Which borough by time combinations have the highest demand, best economics, and worst service gaps?

Stakeholders: Lyft CEO (competitive edge by segment), New Entrant (underserved zones)

Key Technique: RANK() with PARTITION BY

```
RANK() OVER (
  PARTITION BY company_name
  ORDER BY COUNT(*) DESC
) AS volume_rank_within_platform
```

PARTITION BY company_name creates independent rank sequences for Uber and Lyft. Uber ranks its boroughs by volume independently of Lyft. This lets us compare relative importance across platforms Manhattan is both Uber's and Lyft's top borough by volume, but the Bronx is Uber's fourth and Lyft's fifth.

Service Status Labels

```
CASE
  WHEN AVG(sla_5min_met) * 100 < 50 THEN 'UNDERSERVED'
  WHEN AVG(sla_5min_met) * 100 < 58 THEN 'AT RISK'
  ELSE 'ADEQUATE'
END AS service_status
```

50% threshold means less than half of riders get a pickup within 5 minutes. 58% is marginally acceptable. Thresholds are business judgment calls based on NYC urban mobility literature, not TLC mandated values.

Two Part Structure

Part 1 aggregates at Borough by Platform level (10 rows for the two platforms across five boroughs). Part 2 adds the Time of Day dimension up to 50 rows (5 boroughs times 5 time bands times 2 platforms). Part 2 is the full heatmap used in the Power BI Lyft CEO dashboard.

QUERY 03 Origin to Destination Corridor Analysis

Which specific pickup to dropoff corridors generate the most revenue and where is the airport premium?

Stakeholders: Uber CEO (corridor defense), Lyft CEO (inter borough competitive position)

Key Design: Volume Threshold HAVING COUNT(*) >= 50

Zone level pairs with fewer than 50 trips produce statistically unreliable averages. A corridor with 3 trips and avg fare $200 could reflect two wealthy tourists and tells us nothing actionable. 50 trips is a practical minimum for business decision grade metrics at the zone level.

Three Part Structure

- Part 1: Zone level origin to destination pairs, top 20 by trip count, minimum 50 trips each
- Part 2: Airport vs. Non Airport economics by platform uses is_airport_trip flag (airport_fee greater than 0)
- Part 3: Borough to Borough revenue matrix all borough combinations, top 25 by volume

QUERY 04 Take Rate by Fare Decile

Where in the fare spectrum is margin healthy vs. structurally broken?

Stakeholders: Uber CEO (margin leak diagnosis), New Entrant (pricing opportunity identification)

Key Technique: NTILE(10) Window Function

```
NTILE(10) OVER (
  PARTITION BY company_name
  ORDER BY base_passenger_fare
) AS fare_decile
```

NTILE(10) divides each platform's trips into 10 equal size buckets ordered by fare. D1 = cheapest 10%, D10 = most expensive 10%. PARTITION BY company_name gives Uber and Lyft independent decile scales their fare distributions differ so D5 has different dollar values for each platform.

Why NTILE over fixed dollar ranges: fixed ranges ($0 to $10, $10 to $20, etc.) are arbitrary and create unequal bucket sizes. D1 might have 30,000 trips while D9 has 800. NTILE guarantees equal representation in each decile, making all cross decile comparisons statistically equivalent.

CTE Architecture

```
WITH deciled AS (
  -- Step 1: Assign NTILE decile to each row
  SELECT company_name, base_passenger_fare, ...,
    NTILE(10) OVER (PARTITION BY company_name
      ORDER BY base_passenger_fare) AS fare_decile
  FROM v_trips_enriched
),
decile_summary AS (
  -- Step 2: Aggregate per decile
  SELECT company_name, fare_decile, COUNT(*),
    AVG(...), SUM(...)
  FROM deciled
  GROUP BY company_name, fare_decile
)
SELECT * FROM decile_summary
ORDER BY company_name, fare_decile;
```

Two CTE chain. CTEs make the window function result available for aggregation you cannot use NTILE in a GROUP BY directly. The first CTE captures the per row decile assignment; the second CTE aggregates within each decile.

> **Counter Intuitive Result Enabled by This Query**
>
> Uber's negative take rate peaks at D4 to D6 ($13 to $24 fare range), not D1 (cheapest trips). D10 generates 49.7% of all Uber platform margin despite being only 10% of trips. The middle fare range is where Uber's pricing algorithm is most miscalibrated against driver pay obligations.

QUERY 05 Passenger Price Decomposition

What is a rider actually paying for, and are flat fees creating inequitable burdens?

Stakeholders: TLC Commissioner (congestion equity), Lyft CEO (pricing transparency), New Entrant (cost structure baseline)

Structure

Three parts: (1) Full fee decomposition by platform at aggregate level, showing each component as percentage of total charge. (2) Congestion fee burden by borough, showing which boroughs carry the most flat fee burden. (3) Airport vs. Non Airport fee structure comparison.

Key Calculation: Congestion Fee as Percentage of Total

```
ROUND(
  AVG(total_congestion_fees)
  / NULLIF(AVG(passenger_total_charge), 0) * 100
, 2) AS congestion_pct_of_charge
```

This ratio reveals the flat fee burden by trip type. Short trip riders in Manhattan pay 8.9% of their total charge in congestion fees. Long trip riders pay 1.9%. The fee is the same dollar amount but represents a much higher percentage of a cheap short trip fare the structural regressivity.

QUERY 06 Driver Pay Stress Test

Are platforms paying drivers fairly? Where do TLC violations concentrate?

Stakeholders: TLC Commissioner (enforcement priorities), Uber CEO (compliance liability exposure)

Violation Detection Logic

```
-- From the view (precomputed):
below_tlc_minimum = 1 when
  driver_pay < ROUND(
    (trip_miles * 0.82) + (trip_time / 60.0 * 0.57), 2)

-- In Q06 aggregation:
SUM(below_tlc_minimum) AS violation_count,
ROUND(AVG(below_tlc_minimum) * 100, 2) AS violation_rate_pct,
```

```
ROUND(AVG(CASE WHEN below_tlc_minimum = 1
  THEN (trip_miles*0.82 + trip_time/60.0*0.57) - driver_pay
  ELSE 0 END), 2) AS avg_shortfall_usd
```

avg_shortfall_usd is the average dollar amount by which drivers were underpaid on violation trips. The CASE computes shortfall only on violation trips, then AVG is taken across ALL trips in the segment producing a per trip average shortfall across the full segment, not just violation trips.

Multi Dimensional Structure

- Borough level: violation rate by borough and platform
- Time of day level: violation rate by time window and platform
- Distance band level: violation rate by trip distance (0 to 2, 2 to 5, 5 to 10, 10 to 20, 20+ miles)
- Enforcement priorities: combined Borough by Time with both platforms' SLA and violation data

Violations Peak on LONG Trips, Not Short Ones

Uber 5+ mile trips: 2.14% violation rate (worst band). Under 1 mile trips: 0.05%. The TLC minimum pay formula's time component ($0.57/minute) creates underpayment when drivers spend excessive time on congested long routes without proportional fare increases. Short trips have high per minute rates; long congested trips do not.

QUERY 07 Congestion Burden Analysis

Are NYC congestion fees regressive? Who bears the disproportionate flat fee burden?

Stakeholders: TLC Commissioner (equity mandate), Policy analysts

Two Dimensional Analysis

Part 1 examines congestion burden by borough. Tests whether outer boroughs or Manhattan bears more total congestion fees. Result: outer boroughs pay very little ($0.09 to $0.49 per trip) because they rarely cross the Manhattan trigger zones.

Part 2 examines congestion burden by distance band. Tests whether short or long trips bear disproportionate flat fee burden. Result: short trips (0 to 2 miles) pay 8.9% of total charge in congestion fees; 20+ mile trips pay 1.9%. The regressivity is distance based, not borough based.

Key Ratio

```
ROUND(
  SUM(total_congestion_fees)
  / NULLIF(SUM(passenger_total_charge), 0) * 100
, 2) AS congestion_pct_of_total_charge
```

QUERY 08 Holiday Demand Stress Test

How severe is the pre holiday demand surge and post Christmas cliff? Which platform is more resilient?

Stakeholders: Uber CEO (revenue forecasting), Lyft CEO (competitive resilience)

Key Technique: LAG() Window Function

```
LAG(daily_revenue, 1) OVER (
  PARTITION BY company_name
  ORDER BY trip_date
) AS prev_day_revenue,

ROUND(
  (daily_revenue - LAG(daily_revenue,1) OVER (...))
  / NULLIF(LAG(daily_revenue,1) OVER (...), 0) * 100
, 2) AS day_over_day_pct_change
```

LAG(n, 1) retrieves the value from the previous row within the partition. PARTITION BY company_name ensures the LAG does not cross from Lyft's last day to Uber's first day. This creates a day over day revenue change series for each platform, enabling identification of peak and trough dates.

Peak to Trough Calculation

A CTE identifies the daily peak and trough dates per platform. The final SELECT computes peak_to_trough_pct = (trough minus peak) / peak multiplied by 100. Uber: negative 56.1% (Dec 11 to Dec 26). Lyft: negative 44.6% (Dec 12 to Dec 25). Lyft's shallower cliff reflects heavier Manhattan exposure where business travel partially sustains demand through the holiday period.

QUERY 09 Price Volatility Analysis

Which zones have unstable fares that create unpredictable rider experience and driver earnings?

Stakeholders: Lyft CEO (zone strategy), New Entrant (market entry risk assessment)

Coefficient of Variation

```
ROUND(
  STDDEV(base_passenger_fare)
  / NULLIF(AVG(base_passenger_fare), 0) * 100
, 2) AS coeff_of_variation_pct

CASE
  WHEN coeff_of_variation_pct < 35 THEN 'STABLE'
  WHEN coeff_of_variation_pct < 60 THEN 'MODERATE'
  ELSE 'HIGH VOLATILITY'
END AS volatility_label
```

CV is used instead of raw STDDEV because it normalizes for price level. A zone with avg fare $70 and STDDEV $10 (CV 14%) is more predictable than a zone with avg fare $20 and STDDEV $8 (CV 40%), even though its absolute price swing is smaller.

Minimum Volume Filter

```
HAVING COUNT(*) >= 30  -- Minimum for reliable variance calculation
```

Below 30 trips, a single outlier trip dominates STDDEV. The threshold is set conservatively for variance based metrics. Query 10 uses 50 trips as the minimum for the composite opportunity scoring.

QUERY 10 Zone Opportunity Matrix

Which zones are genuinely underserved vs. well defended by incumbents? Where should a new entrant enter?

Stakeholders: New Market Entrant (primary), TLC Commissioner (service gap identification)

Composite Scoring Architecture

```
WITH zone_metrics AS (
  SELECT pu_zone, pu_borough,
    COUNT(*) AS trip_volume,
    AVG(base_passenger_fare) AS avg_fare,
    AVG(sla_5min_met)*100 AS sla_pct,
    AVG(request_to_pickup_min) AS avg_wait
  FROM v_trips_enriched
  GROUP BY pu_zone, pu_borough
  HAVING COUNT(*) >= 50
),
scored AS (
  SELECT *,
    (trip_volume - MIN(trip_volume) OVER ())
    / NULLIF(MAX(trip_volume) OVER ()
      - MIN(trip_volume) OVER (), 0) * 100
    AS volume_score,
    -- SLA GAP: higher score = worse SLA (better entry opportunity)
    GREATEST(0, 70 - sla_pct) AS sla_gap_raw,
  FROM zone_metrics
)
SELECT zone, borough,
  ROUND(
    (volume_score * 0.30) + (fare_score * 0.25)
    + (sla_gap_score * 0.30) + (wait_gap_score * 0.15)
  , 2) AS opportunity_score
FROM scored ORDER BY opportunity_score DESC;
```

All sub scores are min max normalized to 0 to 100 within the query population (zones with 50 or more trips). This makes scores comparable across metrics with different scales. Zones scoring 75 or above are labeled STRONG ENTRY TARGET.

SLA Gap Scoring Logic

The SLA gap score rewards zones where SLA is below the 70% threshold. A zone with 45% SLA receives a higher opportunity score than a zone with 65% SLA because the worse the incumbent service, the bigger the entry opportunity. The 70% threshold is set as the minimum acceptable service standard.

Only 2 STRONG ENTRY TARGET Zones in the Entire Dataset

LaGuardia Airport: score 100.0 (SLA 51%, avg fare $76, high volume, 27.3% WAV match rate). JFK Airport: score 78.6 (SLA 57%, avg fare $63). All other zones score below 65, classified as MONITOR. The airport paradox: riders paying the most receive the worst service the defining entry opportunity.

QUERY 11 Growth Quality Score

Which platform segments have genuinely sustainable growth vs. fragile or loss making operations?

Stakeholders: Uber CEO (segment portfolio review), New Entrant (avoid replicating incumbent mistakes)

Four Dimension Equal Weight Scoring

```
-- Each dimension scored 0 to 25, total maximum = 100

-- Margin score: take rate vs. platform average
CASE
  WHEN take_rate >= platform_avg * 1.2 THEN 25
  WHEN take_rate >= platform_avg       THEN 20
  WHEN take_rate >= 0                  THEN 10
  ELSE 0
END AS margin_score,

-- Service score: SLA performance
CASE
  WHEN sla_pct >= 65 THEN 25
  WHEN sla_pct >= 55 THEN 15
  ELSE 5
END AS service_score,

-- Compliance score: TLC violation rate
CASE
  WHEN violation_rate < 0.5 THEN 25
  WHEN violation_rate < 1.5 THEN 15
  ELSE 5
END AS compliance_score,

-- Stability score: fare coefficient of variation
CASE
  WHEN cv < 35 THEN 25
  WHEN cv < 60 THEN 15
  ELSE 5
END AS stability_score
```

Equal weights (25 each) enforce a no single dimension failure rule. A segment scoring 25+25+25+0 = 75 appears SOLID by composite score but has a compliance problem. Analysts should always review individual dimension scores even when the composite is acceptable.

Segment Classification

Score Range	Classification
80 to 100	SUSTAINABLE strong on all four dimensions
60 to 79	SOLID acceptable performance, minor gaps
40 to 59	FRAGILE material weaknesses in one or two dimensions
Below 40	VULNERABLE failing on multiple dimensions

Lyft 0% Vulnerable, Uber 45.9% Vulnerable

Lyft: 63.4% SOLID, 0% VULNERABLE. Uber: 9.5% SOLID, 45.9% VULNERABLE. Uber Staten Island Late Night: negative take rate, 29.7% SLA, highest relative violation rate, 8.67 minute avg wait. The single worst performing segment in the entire dataset the platform loses money AND delivers terrible service on 175 trips per month.

Section 4 SQL Patterns Used Across Queries

Window Functions Reference

Function	Where Used and Purpose
SUM(COUNT(*)) OVER ()	Q01: Total trips across all platform rows in the same result set. Enables market share calculation in one query without a subquery.
RANK() OVER (PARTITION BY company_name ORDER BY COUNT(*) DESC)	Q02: Ranks boroughs by volume independently within each platform.
NTILE(10) OVER (PARTITION BY company_name ORDER BY base_fare)	Q04: Divides each platform's trips into 10 equal fare buckets.
LAG(daily_revenue, 1) OVER (PARTITION BY company_name ORDER BY trip_date)	Q08: Retrieves previous day's revenue for day over day change calculation.
ROW_NUMBER() OVER (PARTITION BY company_name ORDER BY opportunity_score DESC)	Q10: Ranks zones within each platform for top N filtering.
MIN/MAX(...) OVER ()	Q10 and Q11: Min max normalization of raw metrics to 0 to 100 scale across the full result set.

CTE (Common Table Expression) Pattern

CTEs are used in Q04, Q08, Q10, and Q11. The consistent pattern across all of them:

```
WITH step_one AS (
  -- Compute window functions or intermediate results
  SELECT ...,
    WINDOW_FUNCTION() OVER (...) AS derived
  FROM v_trips_enriched
),
step_two AS (
  -- Aggregate or filter on the derived column
  SELECT ..., AVG(derived), SUM(...)
  FROM step_one
  GROUP BY ...
)
SELECT * FROM step_two ORDER BY ...;
```

This pattern is necessary because MySQL does not allow window functions in WHERE or HAVING clauses, and does not allow aggregation directly on window function results. The CTE materializes the intermediate result, enabling the next step to filter or aggregate on it cleanly.

NULLIF Safety Pattern

```
-- Always wrap division denominators with NULLIF
(SUM(base_fare) - SUM(driver_pay))
/ NULLIF(SUM(base_fare), 0) * 100

-- NULLIF(expr, 0) returns NULL if expr = 0
-- MySQL returns NULL for NULL / anything
-- Result: zero denominator returns NULL instead of a crash
```

Every division in this codebase wraps the denominator in NULLIF(expr, 0). This ensures a zero denominator returns NULL rather than a runtime error. NULL propagates cleanly through AVG and SUM it does not crash the query or produce infinity.